COMP3711 MIDTERM, SIDE A: THEORY — Fri Oct 16 · 20:00–22:00 · LTC (L2) · closed book, this sheet, no calculator · coverage: prerequisites → binary search → mergesort → max subarray → multiplication → randomization → quicksort/selection → heaps → lower bounds & linear sorting · colours: asymptotics recurrences search/D&C sorting randomness heaps lower bounds

A1 · ASYMPTOTICS

In words: O(g) = f grows **no faster** than g (g is a ceiling) · Ω(g) = f grows **at least as fast** as g (floor) · Θ(g) = **same growth rate** · constant factors and small n are ignored, e.g. 5n+3 is O(n²), 3n²+7n is Θ(n²).
Formal. **O(g)**: ∃c,n₀: f(n) ≤ c·g(n) ∀n ≥ n₀ (upper). **Ω(g)**: f ≥ c·g (lower). **Θ(g)**: both (prove each side). **o(g)**: f/g → 0. "Faster" = smaller running time.

Worst case is Θ(g) ⇔ O(g) on every input **and** one input **family** (one per n) costing Ω(g). One single input is not enough.
Transitive; O+O = O; max(f,g) = Θ(f+g). Σ of k terms each O(f) is O(f) only for **fixed** k (T_i = i·n ⇒ g_n = Θ(n³)). T_1,T_2 = O(f) ⇏ T_1/T_2 = O(1).

Growth ladder (slow → fast)

1 < loglog n < log n < (log n)^c < n^ε < n < n log n < n^1.5 < n² < n^c < $2^{\sqrt{n}}$ < 2^n < 3^n < n! < n^n. Polylog < any polynomial; polynomial < any exponential.
10^(10^10) = Θ(1).

Rewrite rules

Log base is irrelevant; log(n^c) = Θ(log n); log means $\log_2$.
a^(log_b n) = n^(log_b a): $3^{(\log_2 n)} = n^{1.585}$ · $4^{(\log_2 n)} = n^2$ · $16^{(\log_4 n)} = n^2$ · $3^{(\log_4 n)} = n^{0.79}$ · $2^{(\log\log n)} = \log n$.
Exponent: **added** constant = same class ($2^{(n+5)}$ = Θ(2^n)); **multiplied** constant = different ($2^{(2n)} = 4^n \neq O(2^n)$; $5^n = o(7^n)$; $\sqrt{(4^n)} = 2^n$; 2^n vs $2^{(n/2)}$: $m = 2^{(n/2)} \Rightarrow m^2$ vs m).
2^f = Θ(2^g) ⇔ lf − gl = O(1). f = Θ(g) is **not** enough (f = 2n, g = n).
n! = Θ(√n (n/e)^n); log(n!) = Θ(n log n); n! ≥ (n/2)^(n/2); n!/(n−2)! = n² − n; n^(1/log n) = 2.
Weird functions: rewrite each as 2^(h(n)) and compare the h's. (log n)^(log n) = n^(loglog n) = 2^(log n·loglog n); n^√log n = $2^{((\log n)^{1.5})}$; $2^{((\log n)^2)}$ is bigger. √log n vs log √n = ½ log n: Ω only. Substitute m = log n or m = √n.

Sums

Σ_{i≤n} i^c = Θ(n^(c+1)): upper = n · max term; lower = **half trick** (top half, each ≥ (n/2)^c). Σ i = n(n+1)/2. Σ i log i = Θ(n² log n). Σ i√n = Θ(n^2.5); Σ i√i = Θ(n^2.5).
$H_n = \Sigma\ 1/i = \Theta(\log n)$ (doubling blocks, each in [½,1]).
Σ_{i=k+1}^{N} 1/i = ln(N/k) + O(1).
Geometric Σ r^i: r < 1 → Θ(1) = 1/(1−r); r > 1 → Θ(last term); r = 1 → count. Σ $2^{(-i)}$ = 2. Σ i/2^i = 2. Σ k x^(k−1) = 1/(1−x)²; Σ k/$2^{(k+1)}$ ≤ 1.
(1−1/n)^n → 1/e ≈ 0.37; (1+1/n)^n → e. Triple loop i ≤ j, k ∈ [i,j]: Θ(n³).

A / B / U decision rule

① simplify each to one term (solve any recurrence or Σ first). ② both Θ, different ⇒ smaller wins. ③ same Θ class ⇒ **U**. ④ one-sided bound: ask "could the hidden value flip the order?" yes ⇒ U; no ⇒ the bound still decides (O(n²) vs Ω(2^n) ⇒ A; O(polylog) vs Ω(n^0.1) ⇒ A). ⑤ O vs O, Ω vs Ω, O(log n) vs Θ(loglog n) ⇒ U.

Solved rows: 4T(n/4)+n = n log n · 5T(n/2)+n² = n^2.32 · 2T(n/2)+n = n log n · 3T(n/2)+n = n^1.585. $\log_3(n!)$ = Θ(n log n) but (log n!)³ = Θ(n³ log³n): read sub vs superscript.

A2 · RECURRENCES

In words: a recurrence is "time for size n = (number of recursive calls) × (time for the smaller size) + (work done in this call)". In T(n) = aT(n/b) + f(n): a = calls, b = how much smaller, f = split + combine work. Mergesort: two sorts of n/2 plus n merging ⇒ 2T(n/2) + n. **Solving** = unroll until the base case and add up what you collected (expansion), or draw the calls as a tree and add the levels (recursion tree); same arithmetic.

Master, T = aT(n/b) + n^d: a > b^d ⇒ **Θ(n^(log_b a))** leaves win · a = b^d ⇒ **Θ(n^d log n)** · a < b^d ⇒ **Θ(n^d)** root wins. Extra log in f ⇒ one more log power ($4T(n/4)+n \log n = n \log^2 n$). Leaves = a^(log_b n) = n^(log_b a).

Expansion (copy this): T(n) = aT(n/b)+f(n) = $a^2T(n/b^2)$+a f(n/b)+f(n) = … = a^k T(n/b^k) + Σ_{i<k} a^i f(n/b^i); stop at k = log_b n, a^k = n^(log_b a). Level i cost a^i f(n/b^i): equal levels ⇒ × log_b n; shrinking ⇒ Θ(f(n)); growing ⇒ Θ(leaves).
Tree table: level i: a^i nodes · size n/b^i · cost/node f(n/b^i) · level total; leaves n^(log_b a)·T(1).
√n: n = 2^m, S(m) = T(2^m): T(√n) → S(m/2). **T(n−1):** expand n−1 levels. **Induction:** guess, prove T(n) ≤ c n^α with the same c.

T(n) =	Θ	T(n) =	Θ
T(n/2)+1	log n	2T(n/2)+n	n log n
2T(n/2)+√n	n	2T(n/2)+n log n	n log² n
3T(n/5)+2n	n	3T(n/4)+n / +n²	n / n²
3T(n/3)+n	n log n	3T(n/2)+n	n^1.585
4T(n/2)+n	n²	4T(n/2)+n²	n² log n
7T(n/2)+n²	n^2.81	8T(n/2)+n²	n³
64T(n/4)+n³	n³ log n	27T(n/3)+n²	n³
9T(n/3)+n²	n² log n	10T(n/3)+n²	n^2.096
4T(n/4)+n $\log_4 n$	n log² n	2T(n/4)+√n	√n log n
2T(n/4)+n	n	T(n/8)+1	log n
2T(√n)+log n	log n·loglog n	T(√n)+log n	log n
T(n/2)+log n	log² n	T(n/2)+n	n (= 2n−1)
T(n−1)+n	n²	2T(n−1)+1	2^n (= 2^n−1)
T(αn)+T(βn)+n, α+β=1	n log n	same, α+β<1	n
T(n/4)+T(3n/4)+n	n log n	T(n/4)+T(3n/4)+1	n
5T(n/3)+n (Toom-3)	n^1.465	7T(n/4)+n	n^1.404

Written-out derivations asked "from first principles": 27T(n/3)+n² = 3^(3k)T(n/3^k) + n²(1+3+…+3^(k−1)) = n³ + n²(n−1)/2 = Θ(n³). · 64T(n/4)+n³: level cost $4^{(3i)}(n/4^i)^3 = n^3$ every level ⇒ n³ + n³ $\log_4 n$ = Θ(n³ log n). · 10T(n/3)+n²: level i = (10/9)^i n², growing ⇒ Θ(n^($\log_3 10$)); write 10^($\log_3 n$) = n^($\log_3 10$). · 4T(n/4)+n $\log_4 n$ (tree): level i: 4^i nodes × (n/4^i) $\log_4(n/4^i) = n(\log_4 n - i)$; leaves 4^($\log_4 n$) = n cost n; total n + n·$\log_4 n(\log_4 n+1)/2$ = Θ(n log² n). · 2T(√n)+log n: n = 2^k, S(k) = 2S(k/2)+k = k log k ⇒ log n·loglog n.

A3 · BINARY SEARCH & DIVIDE AND CONQUER

```
BS(A,l,r,x): if l>r nil; m=⌊(l+r)/2⌋;
A[m]=x→m;
  x<A[m]→BS(l,m-1) else BS(m+1,r)
T=T(n/2)+2 = 2log n+1
LowerBound(x): l=1,r=n+1; while l<r:
m=⌊(l+r)/2⌋;
  if A[m]<x l=m+1 else r=m; return l
(first ≥x; "≤" → first >x
  = successor; n+1 if none)   count(x) =
first>x − first≥x
Rotated (A[k] max): A[m]>A[m+1]→m;
A[m]≥A[l]→right else left;
  find x: x≥A[1]→BS(1,k) else BS(k+1,n)
all O(log n)
Last 0 in 0…01…1: A[m]=0&A[m+1]=1→m;
A[m]=0→right else left;
  k≪n: gallop i=1,2,4,… until A[i]=1, then
FindZ(i/2,i): O(log k)
LastOcc(A,p,r,x) [A[p]=x]: if p=r or A[r]=x
ret r; q=mid;
  A[q]=x&A[q+1]>x→q; A[q]=x→Last(q+1,r);
else Last(p,q-1)
Distinct values (Θ(log n) many): LastOcc
from each block start,
  stop when it returns n → O(log² n)
[S25 Q6, S26 Q3]
```

Design template: Name(input) → output; base case; split; recurse; combine (say exactly how); T(n) = aT(n/b) + split + combine; solve; one line why combine is right.
Target bound ⇒ technique: log² n ⇒ log n binary searches · log k ⇒ halve k · n loglog n ⇒ blocks of size log n · k log k ⇒ heap of size k · n expected ⇒ random pivot, good-pivot stages · n + k log k ⇒ linear pass then sort k · n log k ⇒ k-way heap or recursion depth log k · n log m with Select+Split ⇒ T(n,m) = 2T(n/2,m/2)+n, T(n,1) = n.

- **k-th smallest of two sorted arrays, O(log k)** [F17, F25]: compare A[k/2] and B[k/2]; the smaller one and its prefix have rank < k ⇒ discard them, recurse with k − k/2 (missing entry = ∞); k = 1 ⇒ min of heads. Alternatives: merge until k, Θ(k).
- **Mergesort** 2T(n/2)+n = n log n + n; every input is worst case ⇒ Θ. Merge: two pointers, ∞ sentinels, ties take left (stable), Θ(n).
- **Inversions**: in merge, when R[j] is placed add m−l−i+2 (#left remaining); Θ(n log n). Random permutation has n(n−1)/4 expected inversions; insertion sort = Θ(n + I).
- **Max subarray**: n³ → n² (V(i,j) = V(i,j−1)+A[j]) → n log n (best suffix of left ending at m + best prefix of right from m+1; 2T(n/2)+n) → Kadane O(n): V = max(A[i], V+A[i]). Max product: track max and min. Stock: min of left half + max of right half (n log n), or diffs then Kadane O(n); "no way" if best ≤ 0.
- **Karatsuba**: a = 2^n(n/2)a_1+a_0; ab = 2^nn a_1b_1 + 2^n(n/2) $(a_1b_0+a_0b_1)$ + a_0b_0; middle = $(a_1+a_0)(b_1+b_0) - a_1b_1 - a_0b_0$ ⇒ 3T(n/2)+n = n^1.585 (4 mults ⇒ n²). **Strassen**: 7 products + Θ(n²) adds ⇒ 7T(n/2)+n² = n^2.807 (8 ⇒ n³).
- **Closest pair**: split by x, strip of width 2d sorted by y, ≤7 neighbours ⇒ n log² n (n log n presorted). **Majority**: candidates of halves, count ⇒ n log n; Boyer-Moore O(n) (c=0 adopt, equal c++, else c−−, verify); try-and-check: E ≤ 2 rounds ⇒ O(n). **Grid local min**: border + middle row/col, recurse into smaller quadrant: T(n/2)+n = O(n).
- **k blocks in O(n log k)** [S16, F18, F22]: median via Select + Partition, recurse both halves, stop at size n/k: 2T(n/2)+n with T(n/k) = 1. **k-way merge** [F17]: D&C merges halves, T(k) = 2T(k/2)+kn ⇒ kn log k; or heap of k heads ⇒ kn log k.
- Hanoi 2T(n−1)+1 = 2^n − 1. Subset-sum D&C 2T(n−1)+1 = 2^n. Range query (x,y] count [F22]: sort, LowerBound twice, O(log n). Range-sum O(1) [F22]: D[v] = count, C[i] = i·D[i] + C[i−1], answer C[y] − C[x], build Θ(n+k). Counting-range [F18]: C[i] = #≤ i, answer C[y] − C[x].

A4 · SORTING & SELECTION

```
Partition(A,l,r): x=A[r]; i=l-1; for j=l..r-
1: if A[j]≤x: i++, swap(i,j)
  swap(i+1,r); return i+1.   Inv: A[l..i]≤x
| A[i+1..j-1]>x | ? | x
QS(l,r): m=Partition; QS(l,m-1); QS(m+1,r).
best 2T(n/2)+n; worst
  T(n-1)+n = n² (sorted input, last pivot);
random pivot ⇒ E Θ(n log n)
  on EVERY input; "average" pivot
T(n/4)+T(3n/4)+n = n log n
Select(A,l,r,i): random pivot→m; k=m-l+1;
i=k→A[m]; i<k→left,i;
  else right,i-k.      Fisher-Yates: for
i=1..n swap A[i],A[Random(1,i)]
CountingSort keys 1..k: C[v]++; prefix sums;
for j=n..1: B[C[A[j]]]=A[j];
  C[A[j]]--   (backwards ⇒ stable)  Θ(n+k)
Radix: LSD digit by digit, stable counting
sort per digit, Θ(d(n+k));
  base n ⇒ integers < n^c sort in c passes =
Θ(cn)
```

Quicksort proof: z_1<…<z_n; z_i,z_j compared ⇔ first pivot in {z_i..z_j} is z_i or z_j ⇒ Pr = 2/(j−i+1); E = Σ_{i<j} 2/(j−i+1) ≤ Σ_i 2H_n = O(n log n) (each row O(log n)). **Select proof:** good pivot (middle half) prob ½, removes ≥ ¼; E[tries per good] = 2; ≤ (¾)^i n left after i good; E ≤ Σ 2(¾)^i n = 8n = O(n). Indicator version [Tut 4]: i≤k≤j: 2/(j−i+1); i<j<k: 2/(k−i+1); k<i<j: 2/(j−k+1); each sums O(n). Deterministic median black box ⇒ T(n/2)+n = O(n); weighted median: partition at median, recurse on heavy side.

Sort	Worst	Notes	Space	Stable
Insertion	n²	Θ(n+I); sorted: n; min comparisons on sorted; k-sorted O(nk); online	1	Y
Selection / Bubble	n²	always n² / Θ(n) best with early exit	1	N / Y
Merge	n log n	always; recursive; optimal	n	Y
Heap	n log n	always; in place; optimal; not recursive	1	N
Quick (random)	n²	E n log n; recursive; fastest in practice	log n	N
Counting	n+k	ints in [1,k]	n+k	Y
Radix	d(n+k)	d digits base k	n+k	Y

- **Radix is linear** ⇔ w = log U = O(log n) ⇔ range U = poly(n). T = Θ(⌈w/log k⌉(n+k)), best base k = n. [0,n³): 3 passes Θ(n). [1,$n^{(\ln n)}$]: w = Θ(log² n) ⇒ Θ(n log n); k = U ⇒ super-poly. Range $2^{\Theta(n)}$ ⇒ n²/log n.
- **Fractions p/q ≤ n in O(n)**: gap ≥ 1/n² ⇒ key ⌊n²p/q⌋ ∈ [n,n³], radix base n (no gcd, no floats). **Variable-length strings**: bucket by length, LSD from longest, O(total); padding ⇒ Ω(n²). Bucket sort: uniform [0,1), Θ(n) average. Adjacent-swap sorts need Ω(n²) (one inversion per swap).
- **Pick the sort**: ints in [1,n] ⇒ counting Θ(n) · ints ≤ n² ⇒ radix base n Θ(n) · O(√log n) inversions ⇒ insertion O(n+√log n) · each ≤ d away ⇒ heap of d+1, O(n log d) · stable n log n ⇒ merge · in-place n log n ⇒ heap · general floats ⇒ merge/heap n log n.

COMP3711 MIDTERM, SIDE B: RANDOMNESS · HEAPS · LOWER BOUNDS · PAST-EXAM ANSWER BANK (11 midterms 2015–2026) · EXAM PROTOCOL

B1 · RANDOMIZATION & EXPECTATION

In words: expected value = the long-run average. To find "how many X happen on average", do not compute it directly: split the count into yes/no questions ("does person i get own hat?"), the average of a yes/no question is just its probability, and the average of a sum is the sum of the averages, always, even if the questions depend on each other. Hats: n people × 1/n = 1.

Recipe: ① define X_i = 1 if event i happens (or X_i = waiting time of stage i); ② E[X_i] = Pr[event] (or 1/p); ③ **"by linearity of expectation"** E[ΣX_i] = ΣE[X_i], no independence needed; ④ simplify; exact form when asked. E[XY] = E[X]E[Y] only if independent. E[X] = Σ_{i≥1} Pr[X ≥ i]. Geometric(p): E = 1/p. Randomized time = expected time on the worst input. Bounds both ways: max term up, half trick down.

Classic	Decomposition → answer
Hiring	Pr[i-th hired] = 1/i → H_n = Θ(log n); exactly 1 hire 1/n; n hires 1/n!
Hat check / fixed points / card guessing	each 1/n → **1**; guessing with memory 1/(n–i+1) → H_n
Hat as good or better (1 = best)	Pr = i/n → Σ i/n = **(n+1)/2** [F15, F16]
Coupon, all n	stage i waits n/(n–i) → nH_n = Θ(n log n)
Coupon, first n/2 [S25, S26]	n(1/n + … + 1/(n/2+1)): ≤ n·(n/2)(2/n) = n; ≥ n·(n/2)(1/n) = n/2 → **Θ(n)**, show both
Coupon, last half / last log n	nH_{n/2} = Θ(n log n) / Θ(n loglog n)
Distinct after n boxes [S25, S26]	miss type i: (1–1/n)^n → **n – n(1–1/n)^n** ≈ 0.63n
Empty slots, n balls in n bins	n(1–1/n)^n ≈ n/e; exactly one: n(1–1/n)^(n–1); lonely key (1–1/n)^(n–1)
Birthday / lockers [F18]	pair collides 1/m → C(n,2)/m = n(n–1)/(2m); first pair at n ≈ √(2m)
Hashing, chaining	E[collisions with x] ≤ n/m = α; ops O(1+α); m = Θ(n) ⇒ O(1). Thrown-away [S15]: Pr[i-th thrown] = 1–((n–1)/n)^(i–1)
Local minima, random heap-shaped tree [S19]	root ⅓, internal ¼, leaf ½ → (9n+5)/24
Random-access search [F25]	binary search on a random record; good call (25–75%) prob ½ removes ¼; E[calls per good] = 2; cost ≤ Σ 2(¾)^i n = 8n = O(n)
Uniform {0,1,2} from a fair coin [S16]	2 bits, reject 11; E[flips] = 2/(3/4) = 8/3
Reservoir k of N	kept k/t at step t, k/N final; ever stored k(1+H_N–H_k)

B2 · HEAPS

In words: a min-heap is a binary tree filled level by level, left to right, where every parent is ≤ its children, so the smallest is always on top (siblings in no order). Stored in a plain array, no pointers: position i has its children at 2i and 2i+1. **Formal:** Array H[1..n]: children 2i, 2i+1 · parent ⌊i/2⌋ · complete tree, height ⌊log n⌋ · min at H[1]. **Insert**: append, bubble up, O(log n). **Extract-min**: last → root, sift down (swap with smaller child), O(log n). Find-min O(1). Decrease-key up, increase-key down. **Delete H[i]**: set –∞, bubble to root, extract-min (needs position(x); update it on every swap). **Heapsort**: n log n, in place, not stable, not recursive.

Build-heap O(n): sift down i = ⌊n/2⌋..1; ≤ n/2^(h+1) nodes of height h, cost O(h): Σ h·n/2^(h+1) ≤ n (Σ k x^(k–1) = 1/(1–x)² = 4 at x = ½, times ¼). **F25 version**: pair 2^(m–k) heaps of size 2^k–1 under 2^(m–k–1) new roots, sift each root O(k); Σ_k k2^(m–k–1) = 2^m Σ k/2^(k+1) ≤ n+1. **Drawing** [F25 4,3,8,2,5,6,7,9; S26 5,8,1,4,7,3,2,6]: tree + array after **every** insert; 0.5 off per mistake. Check: filled left to right, parent ≤ children.

d-ary [Tut 5]: children d(i–1)+2..di+1; parent ⌊(j–2)/d⌋+1; height log_d n; insert O(log_d n); extract O(d log_d n); large d when decrease-keys ≫ extracts; d = n ⇒ sorted array.

Sliding-window max [Tut 5]: max-heap of (val,idx), lazy-delete expired roots: heap can reach Θ(n) on increasing input ⇒ O(n log n); with position map ⇒ O(n log W), O(W); monotonic deque O(n) (new item pops smaller from back, pop front if it is A[i–W], front = max). **F22 update(Q,x) O(log w)**: y = front; i = position(y); dequeue; H[i] = –∞; bubble to root; extract-min; insert x; output H[1].

Designs: k-way merge of N items: heap of (value, list, index), size ≤ k, O(N log k), O(k) space · running median: max-heap low, min-heap high, sizes differ ≤ 1, O(log n) insert, O(1) median · optimal merge order = Huffman via heap Θ(n log n) [F18] · k-th smallest in a read-only heap: aux heap, pop and push children, O(k log k) · report all < x: DFS from root, prune at ≥ x, O(k) · d-disordered sort: heap of d+1, O(n log d).

Largest ⌈log n⌉ in O(n loglog n) [S26]: n/log n blocks of log n, sort each (log n·loglog n ⇒ n loglog n total); heap of block tops (n/log n inserts = O(n)); extract-max and push next from that block, log n times = O(log² n).

m sets, findmin O(1), add O(log m + log n_i) [S25]: per-set heap A_i plus top heap H of (min_i, i) with back pointer p_i to its slot; add x: insert into A_i; if x < min_i overwrite H[p_i], bubble up updating p_j on every swap.

B3 · DECISION TREES & LOWER BOUNDS

In words: each comparison is a yes/no question; q questions can tell apart at most 2^q cases. If a problem has N possible answers you need 2^q ≥ N, so q ≥ $\log_2 N$. Sorting has n! answers and log(n!) ≈ n log n, so no comparison sort beats n log n.

Template: ① count the distinct outputs N; ② "any comparison algorithm is a binary decision tree with ≥ N leaves"; ③ "a binary tree of height h has ≤ 2^h leaves, so h ≥ $\log_2 N$"; ④ simplify. Height = worst-case comparisons.

Sorting: n! ⇒ log n! ≥ (n/2) log(n/2) = Ω(n log n). Non-comparison sorts (counting, radix, bucket) never compare keys.

k smallest in order [F25]: C(n,k)·k! = n(n–1)…(n–k+1) ≥ (n/2)^k ⇒ Ω(k log n) for k ≤ n/2.

n/k sorted-block lists of size k [S19]: (k!)^(n/k) outputs ⇒ (n/k) log k! = Ω(n log k). **Do not** argue "each list needs k log k, times n/k": T(P) ≥ max T(P_i) only.

Merging two sorted n-arrays [Tut 5]: C(2n,n) ≥ 2^n ⇒ Ω(n). Max or min: n–1; both ≈ 3n/2.

Median of 3 in ≤ 3 comparisons [S16]: A[1]<A[3]? → A[1]<A[2]? → A[2]<A[3]?; leaves are indices.

Min of 4 recursively [S25]: a_1:a_2, a_3:a_4, then the two winners; 4 leaves per side.

B4 · PAST-EXAM ANSWER BANK

A / B / U rows that appeared (answer, then why)

T_A vs T_B		T_A vs T_B	
n^1.5 vs n²/√log n	A	n² vs 2T(n/2)+n = n log n	B
O(n²) vs Ω(2^n)	A	n²/log n vs n^1.9	B
O(log n) vs Θ(loglog n)	U	10000√n vs 2^n	A
log³n vs ∛n	A	O(√log n) vs Ω(loglog n)	U
O(n²) vs O(n^2.31)	U	5^n vs 7^n	A
2^n vs √(4^n) = 2^n	U	n²/(log n)^10 vs n^1.5	B
Ω(n³) vs O(n^2.81)	B	2^√n vs 10^∛n	B
n² vs 3^($\log_4$ n) = n^0.79	B	n^1.1 vs n^1.2/(log n)³	A
n^2.1 vs n² log³n	B	O(n²) vs O(n^1.9 log n)	U
n^ε vs 2^√n	A	Ω(n²) vs O(n^1.93)	B
O(n²) vs O(n log n)	U	n vs 3^($\log_4$ n)	B
Ω(n³) vs O(n²)	B	Ω(n/log n) vs Ω(10^10)	U
O(n²) vs Θ(n^2.31)	A	n³ vs O(n^$\log_7$ 2)	B
Σ i√n = n^2.5 vs n²	B	4T(n/4)+n vs Σ2^(–i) = Θ(1)	B
O(log(n^10)) vs Ω(n^0.1)	A	n^1.5 $\log_3$ n vs $\log_3$(n!) = n log n	B
2^n vs 2^(n/2)	B	O(log^10(log n)) vs Ω(n log n)	A
O(Σ2^(–i)) vs Ω(log n)	A	O(16^($\log_4$ n)+n^1.5) = O(n²) vs 5T(n/2)+n² = n^2.32	A
Σ i√i = n^2.5 vs n^(4/3)	B	Ω(√(5n)) vs Θ(√5^n)	U

Order-the-functions answers

(log n)² < log(2^n) = 2^(log n) = n < n log n + 1000 < n^1.1 [F17]. 10^(10^10) < $\log^9 n$ < n < n log n < n^1.1/log n [F15]. n^(1/log n) = 2 < H_n < 2^√log n < n²–n < (log n)^(log n) < n^√log n < 2^(log n)² [Tut 1].

Short-answer facts that were marked

- Which sort [F25, S15]: ints in [1,n] ⇒ counting O(n); O(√log n) inversions ⇒ insertion O(n); floats ⇒ merge or heap O(n log n). Insertion over quicksort when: small n, nearly sorted, need stable, online.
- Stable: insertion, merge, counting, radix, bubble, selection as taught. Not stable: quick, heap. Must be recursive: merge, quick. Fewest comparisons on sorted input: insertion. Optimal worst case: merge, heap. Least working space: insertion, heap [F18, S19].
- "Show worst case is Ω(n log n)" = exhibit an input family, one per n [F15]. Insertion sort O(n) only in the best case, so no contradiction with the sorting bound [S15].
- Radix tables [S19, F25]: after each pass rows are sorted by that digit; ties keep the previous order. Radix on variable-length strings padded: Θ(n²) worst [S15].
- Huffman / optimal merge: repeatedly merge the two smallest with a heap, Θ(n log n) [F18]; out of scope otherwise.

B5 · EXAM PROTOCOL

- **Time:** 120 min for 45–60 pts ⇒ about 2 min per point. Order: A/B/U and recurrence (10 min) → heap drawing (10 min) → design questions → expectation.
- **Every design answer:** ① one-line idea; ② "Procedure Name(inputs) → output, precondition"; ③ pseudocode or numbered steps; ④ black boxes named with their time ("Select from class, Θ(n)", "Merge Θ(n)", "binary search O(log n)"); ⑤ recurrence or sum, solved, final Θ; ⑥ one line why correct.
- **Running-time sentence:** "each call does O(f) work and the recursion depth is …, so …" or "T(n) = … which solves to …".
- **A/B/U:** write the simplified form next to each entry so partial credit is visible. **Heap:** array under every tree. **Expectation:** exact formula, no ≈.
- **Never blank:** write the recurrence even if unsolved; write the obvious O(n log n) sort-based answer then improve.

Pitfalls: give Θ with α, β when asked · include the leaf level · no Master theorem when forbidden · define subroutine I/O · justify every running time · never assume independence · both bounds when asked · U only when a one-sided bound truly leaves it open · (1–1/n)^n vs ^(n–1) (self excluded ⇒ n–1) · log = $\log_2$ · "faster" = smaller time.

Math facts: log_b a = log a/log b = 1/log_a b · a^(log_b c) = c^(log_b a) · C(n,k) ≤ n^k/k! · C(2n,n) ≥ 2^n · n! ≈ √(2πn)(n/e)^n · Σ_{i=0}^{k–1} r^i = (r^k–1)/(r–1) · 1+2+…+2^(k–1) = 2^k–1 · e^x ≥ 1+x · (1–p)^k ≤ e^(–pk) · H_n ≈ ln n + 0.577 · $\log_2 10$ ≈ 3.32 · $\log_2 3$ ≈ 1.585 · $\log_2 5$ ≈ 2.32 · $\log_2 7$ ≈ 2.807 · $\log_3 10$ ≈ 2.096 · $\log_4 3$ ≈ 0.79 · 2^10 ≈ 10³.